Technical Documentation

RAG-Based Customer Support Assistant using LangGraph and HITL

1. Introduction

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with language generation to produce accurate and context-aware responses. Instead of relying only on pre-trained knowledge, the system retrieves relevant information from a knowledge base and uses it to generate answers.

In this project, RAG is applied to a customer support use case. Users can upload documents (PDF/TXT), and the system answers queries based on those documents. This ensures that responses are grounded in actual data rather than assumptions.

2. Why RAG is Needed

Traditional chatbot systems often generate incorrect or generic responses because they do not have access to domain-specific data.

RAG improves this by:

- Retrieving relevant information before generating answers
- Reducing hallucinations
- Allowing the system to adapt to new documents dynamically

This makes the system more reliable and suitable for real-world applications.

3. Use Case Overview

The system is designed as a customer support assistant that:

- Accepts documents as knowledge sources
- Answers user queries based on those documents
- Escalates queries when confidence is low

It can be used for:

- FAQ automation
- Product documentation support
- Internal knowledge systems

4. System Architecture Explanation

The system consists of multiple components working together:

- Document ingestion pipeline processes uploaded files
- Embedding model converts text into vectors
- Vector database stores embeddings
- Retriever fetches relevant content
- LLM generates responses
- LangGraph manages workflow
- HITL handles escalation

The architecture ensures a smooth flow from input to output while maintaining accuracy and control.

5. Design Decisions

Chunk Size

A chunk size of around 800 characters with overlap is used to balance context retention and retrieval efficiency.

Embedding Strategy

Sentence-transformer models are chosen because they provide good semantic understanding with low computational cost.

Retrieval Approach

Top-k similarity search is used to retrieve the most relevant chunks from the vector database.

Prompt Design

The prompt is structured to:

- Use only retrieved context
- Avoid hallucination
- Provide clear and professional responses

6. Workflow Explanation (LangGraph)

The system uses LangGraph to implement a structured workflow.

Nodes:

- Retrieve Node → fetches context
- Generate Node → creates answer
- Decision Node → evaluates confidence
- Human Node → handles escalation

Flow:

The query moves through these nodes in sequence. Based on confidence, the system either returns the answer or escalates it.

7. Conditional Logic

The system uses simple decision rules:

- If relevant context is found → generate answer
- If confidence is high → return answer
- If confidence is low → escalate

👉
The system uses LangGraph to implement a stateful workflow with conditional routing, enabling dynamic decision-making and Human-in-the-Loop escalation.

8. HITL Implementation

Human-in-the-Loop (HITL) is implemented as a fallback mechanism.

When triggered:

- Low confidence
- No relevant context
- Complex queries

What happens:

- System avoids giving incorrect answers
- Returns escalation message
- Can be extended to real human support

Benefits:

- Improves trust

- Ensures reliability

9. Challenges & Trade-offs

Retrieval Accuracy vs Speed

More context improves accuracy but increases response time.

Chunk Size vs Context Quality

Smaller chunks improve precision, while larger chunks provide better context.

Cost vs Performance

Larger models give better responses but are more expensive and slower.

10. Testing Strategy

The system is tested using different types of queries:

Sample Queries:

- What is refund policy?
- Summarize the document
- Ask unrelated question